# Restaurant - Case Study

Loading a file to Hadoop:

```
[cloudera@quickstart ~]$ hadoop fs -ls /jeeva
[cloudera@quickstart ~]$ hadoop fs -put '/home/cloudera/Desktop/restaurant.csv' /jeeva
[cloudera@quickstart ~]$ hadoop fs -ls /jeeva
Found 1 items
-rw-r--r--   1 cloudera supergroup    2257316 2026-03-21 00:19 /jeeva/restaurant.csv
```

Reading a file:

```
>>> from pyspark.sql import SparkSession
>>> spark=SparkSession.builder.appName('MyApp').getOrCreate
>>> data=spark.read.csv('/jeeva/restaurant.csv',header=True,inferSchema=True)
```

## 1. Global Cuisine Popularity Analysis

- **The Problem:** The Cuisines column contains comma-separated values (e.g., "French, Japanese, Desserts").
- **Goal:** Calculate the total count of each cuisine globally and per city to identify the most popular food types.

```
>>> from pyspark.sql.functions import *
>>> data=data.withColumn("Cuisine", explode(split("Cuisines", ",")))
>>> data=data.withColumn("Cuisine", trim(data.Cuisine))
```

**Total count of each cuisine globally:**

```
>>> popularity=data.groupBy("Cuisine").agg(count('*').alias('count'))\
... .orderBy('count',ascending=False)
>>> popularity.show()
+------------+-----+
|     Cuisine|count|
+------------+-----+
|North Indian| 3958|
|     Chinese| 2733|
|   Fast Food| 1984|
|     Mughlai|  994|
|     Italian|  763|
|      Bakery|  745|
| Continental|  736|
|        Cafe|  703|
|    Desserts|  653|
|South Indian|  635|
| Street Food|  562|
|    American|  389|
|      Mithai|  380|
|       Pizza|  379|
|      Burger|  250|
|        Thai|  234|
|       Asian|  233|
|   Beverages|  229|
|   Ice Cream|  226|
|     Mexican|  180|
+------------+-----+
```

**Count of each cuisine per city:**

```
>>> city_popularity=data.groupBy("City","Cuisine").agg(count('*').alias('count'))\
... .orderBy('count',ascending=False)
>>> city_popularity.show()
+---------+------------+-----+
|     City|     Cuisine|count|
+---------+------------+-----+
|New Delhi|North Indian| 2425|
|New Delhi|     Chinese| 1638|
|New Delhi|   Fast Food| 1303|
|New Delhi|     Mughlai|  628|
|    Noida|North Indian|  530|
|  Gurgaon|North Indian|  508|
|New Delhi|      Bakery|  465|
|New Delhi|South Indian|  411|
|New Delhi| Street Food|  411|
|New Delhi|    Desserts|  383|
|    Noida|     Chinese|  382|
|New Delhi|     Italian|  376|
|New Delhi| Continental|  373|
|New Delhi|        Cafe|  325|
|  Gurgaon|     Chinese|  324|
|New Delhi|      Mithai|  282|
|    Noida|   Fast Food|  265|
|  Gurgaon|   Fast Food|  220|
|New Delhi|       Pizza|  196|
|    Noida|     Mughlai|  155|
+---------+------------+-----+
```

## 2. Normalizing and Comparing Costs Across Currencies

- **The Problem:** The Average Cost for two is in different local currencies (INR, Dollar, Botswana Pula, etc.).
- **Goal:** Determine which city/country has the most expensive or affordable dining scenes on a global scale.

```
>>> rates=spark.createDataFrame([\
... ("Indian Rupees(Rs.)",1.0),\
... ("Dollar($)",83.0),\
... ("Botswana Pula(P)",6.2)\
... ],["Currency","rate"])
>>> inr_data=data.join(rates,on='Currency')
>>> inr_data=inr_data.withColumn('inr',col('Average cost for two')*col('rate'))
```

**The most expensive dining scenes by city :**

```
>>> inr_data.groupBy('city').agg(avg('inr')).orderBy('avg(inr)').show()
+--------------+------------------+
|          city|          avg(inr)|
+--------------+------------------+
|     Faridabad|476.65330661322645|
|     Allahabad| 512.7659574468086|
|      Amritsar| 545.7142857142857|
|        Mohali|             550.0|
|      Varanasi| 565.7142857142857|
|        Bhopal|            566.25|
|   Dicky Beach|             581.0|
|Lakes Entrance|             581.0|
|     Inverloch|             581.0|
|         Noida| 607.0607175712971|
|     Ghaziabad| 673.4042553191489|
|     New Delhi| 681.9750137791659|
|    Aurangabad| 700.9615384615385|
|  Bhubaneshwar| 721.2962962962963|
|        Nashik| 722.7272727272727|
|        Ranchi| 727.8846153846154|
|      Dehradun| 745.6521739130435|
|       Gurgaon| 784.4960267670431|
|        Nagpur|             787.5|
|         Kochi| 794.8275862068965|
+--------------+------------------+
```

**The most expensive dining scenes by country :**

```
>>> inr_data.groupBy('country code').agg(avg('inr')).orderBy('avg(inr)').show()
+------------+------------------+
|country code|          avg(inr)|
+------------+------------------+
|           1| 706.8202196816857|
|          14|1756.5510204081634|
|         216| 2247.760180995475|
|          37|          3475.625|
|         162|11401.111111111111|
|         184| 12407.65306122449|
+------------+------------------+
```

## 3. Impact of Services on Customer Ratings

- **The Problem:** Does having "Online Delivery" or "Table Booking" actually lead to better reviews?
- **Goal:** Provide business insights into whether investing in delivery infrastructure correlates with higher customer satisfaction.

**Online Delivery vs Rating :**

```
>>> data_rating=data.withColumn('rating',col('Aggregate Rating').cast('double'))
>>> data_rating.groupBy('Has Online delivery')\
... .agg(avg('rating'),count('*').alias('restaurants'))\
... .show()
+------------------+------------------+-----------+
|Has Online delivery|       avg(rating)|restaurants|
+------------------+------------------+-----------+
|                No|2.4656236786469345|       7099|
|               Yes|3.2486938775510255|       2450|
+------------------+------------------+-----------+
```

**Table Booking vs Rating:**

```
>>> data_rating.groupBy('Has Table Booking')\
... .agg(avg('rating'),count('*').alias('restaurants'))\
... .show()
+-----------------+------------------+-----------+
|Has Table Booking|       avg(rating)|restaurants|
+-----------------+------------------+-----------+
|               No|  2.55956837963515|       8391|
|              Yes|3.4419689119170966|       1158|
+-----------------+------------------+-----------+
```

**(Online Delivery and Table Booking) vs Rating:**

```
>>> data_rating.groupBy('Has Online Delivery','Has Table Booking')\
... .agg(avg('rating'),count('*').alias('restaurants'))\
... .show()
+------------------+-----------------+------------------+-----------+
|Has Online Delivery|Has Table Booking|       avg(rating)|restaurants|
+------------------+-----------------+------------------+-----------+
|               Yes|              Yes|3.5958620689655163|        435|
|                No|               No|2.3653483992467024|       6376|
|               Yes|               No|3.1737468982630337|       2015|
|                No|              Yes|3.3493775933609933|        723|
+------------------+-----------------+------------------+-----------+
```

## 4. Identifying "Hidden Gems" (Weighted Rating Analysis)

- **The Problem:** Some restaurants have a 5.0 rating but only 2 votes, while others have 4.5 with 2,000 votes.
- **Goal:** Find highly-rated restaurants that have significant customer trust.

```
>>> data_rating=data_rating.withColumn('vote',col('Votes').cast('int'))
>>> data_rating=data_rating.withColumn('score',col('vote')*col('rating'))
```

```
>>> data_rating.select("Restaurant Name", "City", "rating", "votes", "score")\
... .orderBy(col('score'),ascending=False).show()
+--------------------+---------+------+-----+------------------+
|     Restaurant Name|     City|rating|votes|             score|
+--------------------+---------+------+-----+------------------+
|                Toit|Bangalore|   4.8|10934|           52483.2|
|            Truffles|Bangalore|   4.7| 9667|           45434.9|
|    Hauz Khas Social|New Delhi|   4.3| 7931|34103.299999999996|
|           Peter Cat|  Kolkata|   4.3| 7574|32568.199999999997|
|AB's - Absolute B...|Bangalore|   4.6| 6907|31772.199999999997|
|     Barbeque Nation|  Kolkata|   4.9| 5966|           29233.4|
|AB's - Absolute B...|Hyderabad|   4.9| 5434|26626.600000000002|
|          Big Brewsky|Bangalore|   4.5| 5705|           25672.5|
|           Big Chill|New Delhi|   4.5| 4986|           22437.0|
|     Saravana Bhavan|New Delhi|   4.3| 5172|           22239.6|
|               BarBQ|  Kolkata|   4.2| 5288|22209.600000000002|
|     The Black Pearl|Bangalore|   4.1| 5385|22078.499999999996|
|        Joey's Pizza|   Mumbai|   4.0| 5145|           20580.0|
|              Gulati|New Delhi|   4.4| 4373|           19241.2|
|          Farzi Cafe|  Gurgaon|   4.3| 4385|           18855.5|
|             Karim's|New Delhi|   4.0| 4689|           18756.0|
|      Warehouse Cafe|New Delhi|   3.7| 4914|           18181.8|
|               Ricos|New Delhi|   4.3| 4085|           17565.5|
|     Big Yellow Door|New Delhi|   4.3| 3986|           17139.8|
|     Barbeque Nation|  Chennai|   4.4| 3848|           16931.2|
+--------------------+---------+------+-----+------------------+
```

## 5. Restaurant Clustering for Marketing

- **The Problem:** Grouping restaurants for targeted advertisements.
- **Goal:** Categorize restaurants into segments like "Luxury Dining," "Budget Eats," and "Popular Mid-range."

```
>>> data_market=data.withColumn('cost',col('Average Cost for two').cast('double'))\
... .withColumn('rating',col('Aggregate Rating').cast('double'))\
... .withColumn('vote',col('Votes').cast('int'))
>>> data_market=data_market.withColumn('category',\
... when((data_market.rating>=4.2)&(data_market.cost>=3000),'Luxury Dining')\
... .when((data_market.rating>=4)&(data_market.cost<1000),'Budget Eats')\
... .when((data_market.vote>200)&(data_market.cost.between(1000,3000)),'Popular Mid-range')\
... .otherwise('others')
... )
```

```
>>> data_market.groupBy('category').count().show()
+-----------------+-----+
|         category|count|
+-----------------+-----+
|      Budget Eats|  905|
|Popular Mid-range|  639|
|           others| 7968|
|    Luxury Dining|   37|
+-----------------+-----+
```

## 6. Geolocation-based Food Hub Detection

- **The Problem:** Identifying dense clusters of restaurants (Food Hubs) without relying on locality names.
- **Goal:** Identify the exact geographic areas where restaurant competition is highest

```
>>> geo_data=data.withColumn('lon',round(col('Longitude').cast('double'),2))\
... .withColumn('lat',round(col('Latitude').cast('double'),2))
```

```
>>> geo_data.groupBy('lon','lat').agg(count('*').alias('restaurants'))\
... .orderBy('restaurants',ascending=False).show()
+-----+-----+-----------+
|  lon|  lat|restaurants|
+-----+-----+-----------+
|77.22|28.63|        147|
|77.32|28.57|        104|
|77.33|28.57|         79|
|77.17|28.59|         76|
|77.18|28.64|         72|
|77.21|28.56|         72|
|77.12|28.65|         71|
|77.09|28.49|         70|
|77.25|28.55|         64|
|77.22|28.53|         63|
|77.23|28.57|         61|
|77.25|28.54|         61|
|77.24|28.58|         59|
|77.11|28.64|         54|
|77.29|28.64|         53|
|77.28|28.63|         53|
|77.08|28.46|         52|
|77.15|28.69|         51|
|77.12|28.64|         51|
| 77.1|28.64|         51|
+-----+-----+-----------+
```

## 7. Data Quality & Standardization Pipeline

- **The Problem:** The dataset contains null values in Cuisines and potential encoding issues in city names.
- **Goal:** Ensure the data is "Analytics Ready" for downstream business intelligence tools.

```
>>> data=data.dropna()
>>> data=data.dropDuplicates()
>>> data=data.withColumn(\
... "cost",col("Average Cost for two").cast("double"))\
... .withColumn("rating",col("Aggregate rating").cast("double"))\
... .withColumn("lon",col("Longitude").cast("double"))\
... .withColumn("lat",col("Latitude").cast("double"))\
... .withColumn("price_range",col("Price range").cast("int"))
>>> data=data.withColumn("Cuisine", explode(split("Cuisines", ",")))
>>> data=data.withColumn("Cuisine", trim(data.Cuisine))
```

## 8. Locality-wise Price Range Distribution

- **The Problem:** Analyzing the economic profile of different neighborhoods.
- **Goal:** Generate a report showing the percentage of "Cheap" vs "Expensive" restaurants in every locality to help new entrepreneurs find market gaps.

```
>>> loc_wise=data.withColumn("price_category",\
... when(col('price_range')>2,"Expensive")\
... .otherwise('cheap'))
>>> loc_grp=loc_wise.groupBy('Locality','price_category').count()
>>> loc_cnt=data.groupBy('Locality').agg(count('*').alias('total'))
```

```
>>> grp=loc_grp.join(loc_cnt,on='Locality')
>>> grp.withColumn("percentage",\
... round(col('count')/col('total')*100,2)).orderBy('Locality').show()
+--------------------+--------------+-----+-----+----------+
|            Locality|price_category|count|total|percentage|
+--------------------+--------------+-----+-----+----------+
| ILD Trade Centre...|         cheap|    2|    2|     100.0|
|12th Square Build...|     Expensive|    1|    1|     100.0|
|A Hotel, Gurdev N...|         cheap|    1|    1|     100.0|
|ARSS Mall, Paschi...|         cheap|    1|    1|     100.0|
|           Aaya Nagar|         cheap|    1|    1|     100.0|
|Abu Dhabi Mall, T...|     Expensive|    2|    2|     100.0|
|          Abu Shagara|     Expensive|    3|    3|     100.0|
|Acropolis Mall, K...|     Expensive|    2|    2|     100.0|
|           Adajan Gam|         cheap|    1|    3|     33.33|
|           Adajan Gam|     Expensive|    2|    3|     66.67|
|              Adchini|     Expensive|    5|   13|     38.46|
|              Adchini|         cheap|    8|   13|     61.54|
|       Addition Hills|     Expensive|    1|    1|     100.0|
|Aditya Mega Mall,...|         cheap|    3|    4|      75.0|
|Aditya Mega Mall,...|     Expensive|    1|    4|      25.0|
|                Adyar|         cheap|    1|    1|     100.0|
|             Aerocity|         cheap|    4|    4|     100.0|
|          Aggar Nagar|     Expensive|    1|    1|     100.0|
|Aggarwal City Mal...|         cheap|    1|    3|     33.33|
|Aggarwal City Mal...|     Expensive|    2|    3|     66.67|
+--------------------+--------------+-----+-----+----------+
```

## 9. Price Range Prediction

- **The Problem:** Can we predict the Price range (1 to 4) of a new restaurant based on its location and cuisine?
- **Goal:** Build a model that predicts the category of a restaurant based on its basic profile.

```
>>> data_predict=data.withColumn("predict_price_range",\
... when(col('cost')>=3000,4)\
... .when(col('cost')>=1500,3)\
... .otherwise(1))
```

```
>>> data_predict.select('price_range','predict_price_range').show()
+-----------+-------------------+
|price_range|predict_price_range|
+-----------+-------------------+
|          1|                  1|
|          2|                  1|
|          2|                  1|
|          3|                  1|
|          2|                  1|
|          2|                  1|
|          1|                  1|
|          3|                  3|
|          4|                  3|
|          4|                  3|
|          2|                  1|
|          2|                  1|
|          1|                  1|
|          1|                  1|
|          2|                  1|
|          1|                  1|
|          1|                  1|
|          2|                  1|
|          2|                  1|
|          2|                  1|
+-----------+-------------------+
```

## 10. Top-N Recommendation Engine

- **The Problem:** Recommending similar restaurants to a user.
- **Goal:** Given a restaurant the user likes, recommend 5 other similar restaurants in the same city.

```
>>> liked_res="Vikings"
>>> liked=data.filter(col('Restaurant Name')==liked_res).collect()[0]
>>> liked_cuisines=[i.strip() for i in liked['Cuisines'].split(",")]
>>> filtered=data.filter(\
... (col('Restaurant Name')!=liked_res)&\
... (col('City')==liked['City']))
```

```
>>> filtered.filter(col('Cuisine').isin(liked_cuisines))\
... .groupBy('Restaurant Name')\
... .agg(max('Cuisines').alias("Cuisines"),count('*').alias("Similar Cuisines"))\
... .orderBy(count("*").desc()).show(5,truncate=False)
+----------------------------+---------------------------------------+----------------+
|Restaurant Name             |Cuisines                               |Similar Cuisines|
+----------------------------+---------------------------------------+----------------+
|Heat - Edsa Shangri-La      |Seafood, Asian, Filipino, Indian       |4               |
|Buffet 101                  |Asian, Indian, European                |3               |
|Spiral - Sofitel Philippine |European, Asian, Indian                |3               |
|NIU by Vikings              |Seafood, Indian, Mediterranean, Japanese|2               |
|The Food Hall by Todd English|American, Indian, Italian, Seafood     |2               |
+----------------------------+---------------------------------------+----------------+
```

## 11. Optimized Currency Conversion (Broadcast Join)

- **The Problem**: Joining a massive restaurant dataset with a tiny 15-row currency exchange table triggers a standard shuffle join, moving millions of rows across the network unnecessarily
- **Goal**: Implement a Broadcast Join to eliminate network shuffle and speed up the cost normalization process across different global currencies.

```
>>> rates=spark.createDataFrame([\
... ("Indian Rupees(Rs.)",1.0),\
... ("Dollar($)",83.0),\
... ("Botswana Pula(P)",6.2)\
... ],["Currency","rate"])
>>> from pyspark.sql.functions import broadcast
>>> inr_data=data.join(broadcast(rates),on="Currency")
```

## 12. Efficient Data Pipelines (Caching & Storage)

- **The Problem:** The "Cuisine Popularity" analysis and "Price Range Prediction" both use the same cleaned dataset. Without optimization, Spark re-reads the raw CSV from HDFS and re-calculates the split() and explode() transformations for every single query.
- **Goal:** Use **.cache()** to persist intermediate cleaned data in memory and save the final "Analytics Ready" dataset in **Parquet** format to leverage columnar storage and compression.

```
>>> data_cleaned=data.withColumn("Cuisine",explode(split("Cuisines",",")))\
... .withColumn("Cuisine",trim(col("Cuisine")))
```

```
>>> data_cleaned.cache()
2026-03-23 21:49:47 WARN  CacheManager:66 - Asked to cache already cached data.
DataFrame[Restaurant ID: string, Restaurant Name: string, Country Code: string,
City: string, Address: string, Locality: string, Locality Verbose: string, Longi
tude: string, Latitude: string, Cuisines: string, Average Cost for two: string,
Currency: string, Has Table booking: string, Has Online delivery: string, Is del
ivering now: string, Switch to order menu: string, Price range: string, Aggregat
e rating: string, Rating color: string, Rating text: string, Votes: int, Cuisine
: string]
>>> data_cleaned.count()
19704
>>> data_cleaned=data_cleaned.toDF(*[\
... c.replace(" ", "_") for c in data_cleaned.columns])

>>> data_cleaned.write.mode("overwrite").parquet("/cleaned/")

>>> new_data=spark.read.parquet("/cleaned/")
>>> new_data.select("Cuisine").show()
+---------+
|  Cuisine|
+---------+
|   French|
| Japanese|
| Desserts|
| Japanese|
|  Seafood|
|    Asian|
| Filipino|
|   Indian|
|   Indian|
|    Sushi|
| Japanese|
|   Indian|
```

## 13. Service Impact Comparative Visualization

- **The Problem:** Simply listing the average rating for restaurants with "Online Delivery" vs. "Table Booking" doesn't show the full story of customer satisfaction or the volume of feedback.
- **Goal:** Create a **Side-by-Side Box Plot** to compare rating distributions for each service, using **Color Overlays** to represent different price ranges and **Point Density (Jitter)** to show the total number of votes.

```
import numpy as np

df["Service"] = np.where(

   (df["Has Online delivery"] == "Yes") & (df["Has Table booking"] == "Yes"),
```

```
    "Both",
    np.where(
        df["Has Online delivery"] == "Yes",
        "Online Delivery",
        np.where(
            df["Has Table booking"] == "Yes",
            "Table Booking",
            "None"
        )
    )
)
# Filter
df = df[df["Service"] != "None"]
import seaborn as sns
import matplotlib.pyplot as plt
plt.figure(figsize=(12,6))
# Box Plot (distribution of ratings)
sns.boxplot(
    x="Service",
    y="Aggregate rating",
    hue="Price range",
    data=df
)
# Jitter points (votes density)
```

```
sns.stripplot(
    x="Service",
    y="Aggregate rating",
    data=df,
    color="black",
    size=3,
    jitter=True,
    alpha=0.3
)
plt.title("Service Impact on Ratings with Price Range & Vote Density")
plt.legend(title="Price Range", bbox_to_anchor=(1.05, 1))
plt.show()
```